

Varsha M

TASK 4 · Unveiling the Android App Market

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
sns.set_style("whitegrid")
```

```
df = pd.read_csv("Google-Playstore.csv")
print("Dataset Shape:", df.shape)
df.head()
```

Dataset Shape: (21489, 24)

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	...	Dev
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	10+	10.0	15	True	0.0	...	https://l
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0	7662	True	0.0	...	https://webs
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0	58	True	0.0	...	
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0	19	True	0.0	...	http://www.clime
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	100+	100.0	478	True	0.0	...	http://www

5 rows × 24 columns

```
print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])
print("\nColumn Names:")
print(df.columns.tolist())
print("\nData Types:")
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
```

Number of Rows: 21489
Number of Columns: 24

Column Names:
['App Name', 'App Id', 'Category', 'Rating', 'Rating Count', 'Installs', 'Minimum Installs', 'Maximum Installs', 'Free', 'Price', 'Currency', 'Size', 'Minimum Android', 'Developer Id', 'Developer Website']

Data Types:
App Name object
App Id object
Category object
Rating float64
Rating Count float64
Installs object
Minimum Installs float64
Maximum Installs int64
Free bool
Price float64
Currency object
Size object
Minimum Android object
Developer Id object
Developer Website object

```

Last Updated      object
Content Rating    object
Privacy Policy    object
Ad Supported      object
In App Purchases  object
Editors Choice    object
Scraped Time      object
dtype: object

```

Missing Values:

```

App Name          0
App Id            0
Category          0
Rating            186
Rating Count      186
Installs          1
Minimum Installs  1
Maximum Installs  0
Free              0
Price             0
Currency          1
Size              0
Minimum Android   62
Developer Id      0
Developer Website 7037
Developer Email   0
Released          609
Last Updated      1
Content Rating    1
Privacy Policy    3844
Ad Supported      1
In App Purchases 1
Editors Choice    1
Scraped Time      1
dtype: int64

```

Cleaning

```

df = df.drop_duplicates(subset='App Name')
print("Shape after removing duplicates:", df.shape)

```

Shape after removing duplicates: (21437, 24)

```
df['Rating'] = pd.to_numeric(df['Rating'], errors='coerce')
```

```
df['Rating Count'] = pd.to_numeric(df['Rating Count'], errors='coerce')
```

```

df['Installs'] = (
    df['Installs'].astype(str).str.replace(',', '', regex=False).str.replace('+', '', regex=False)
df['Installs'] = pd.to_numeric(df['Installs'], errors='coerce')

```

```
df['Maximum Installs'] = pd.to_numeric(df['Maximum Installs'], errors='coerce')
```

```
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
```

handle missing values

```

df['Rating'] = df['Rating'].fillna(df['Rating'].median())
df['Rating Count'] = df['Rating Count'].fillna(0)
df['Price'] = df['Price'].fillna(0)
df['Installs'] = df['Installs'].fillna(0)
df['Maximum Installs'] = df['Maximum Installs'].fillna(0)

```

```
print(df.isnull().sum())
```

```

App Name          0
App Id            0
Category          0
Rating            0
Rating Count      0
Installs          0
Minimum Installs  1
Maximum Installs  0
Free              0

```

```

Price          0
Currency       1
Size           0
Minimum Android 62
Developer Id    0
Developer Website 7010
Developer Email 0
Released       609
Last Updated   1
Content Rating 1
Privacy Policy 3830
Ad Supported    1
In App Purchases 1
Editors Choice 1
Scraped Time    1
dtype: int64

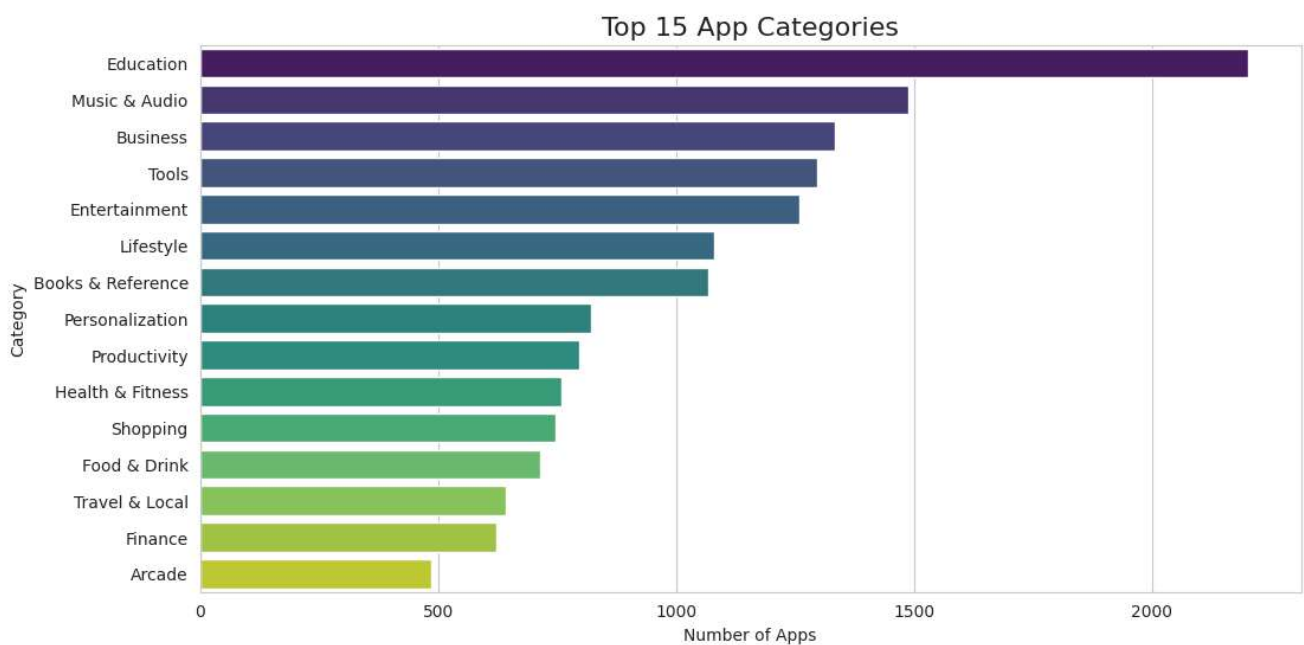
```

category analysis

```

category_counts = df['Category'].value_counts().head(15)
plt.figure(figsize=(12,6))
sns.barplot(x=category_counts.values,y=category_counts.index,hue=category_counts.index,palette='viridis',legend=False)
plt.title("Top 15 App Categories", fontsize=16)
plt.xlabel("Number of Apps")
plt.ylabel("Category")
plt.show()

```

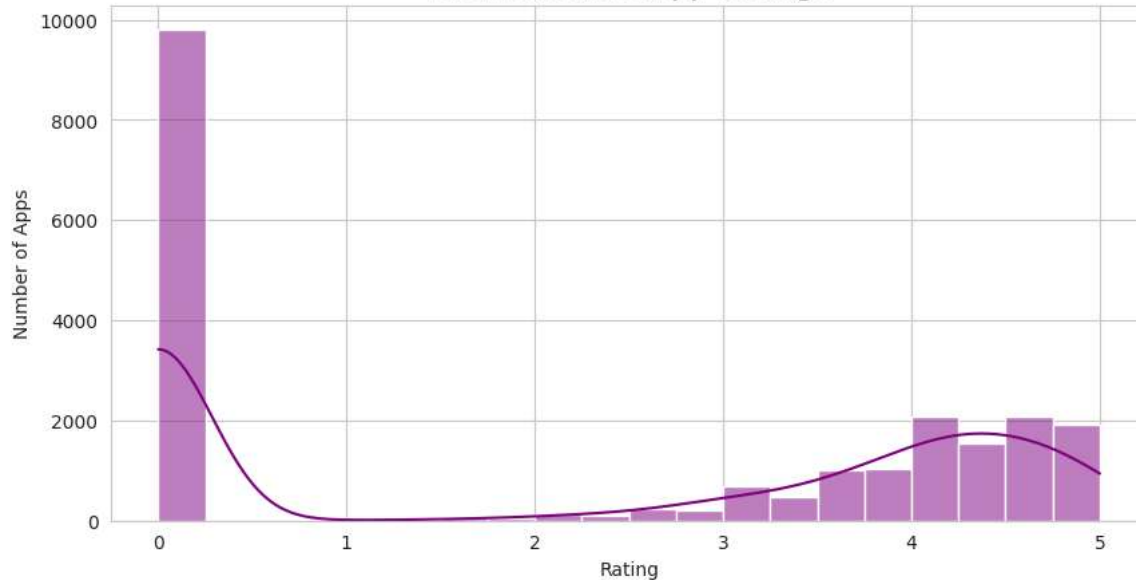


```

plt.figure(figsize=(10,5))
sns.histplot(df['Rating'].dropna(),bins=20,kde=True,color='purple')
plt.title("Distribution of App Ratings", fontsize=16)
plt.xlabel("Rating")
plt.ylabel("Number of Apps")
plt.show()

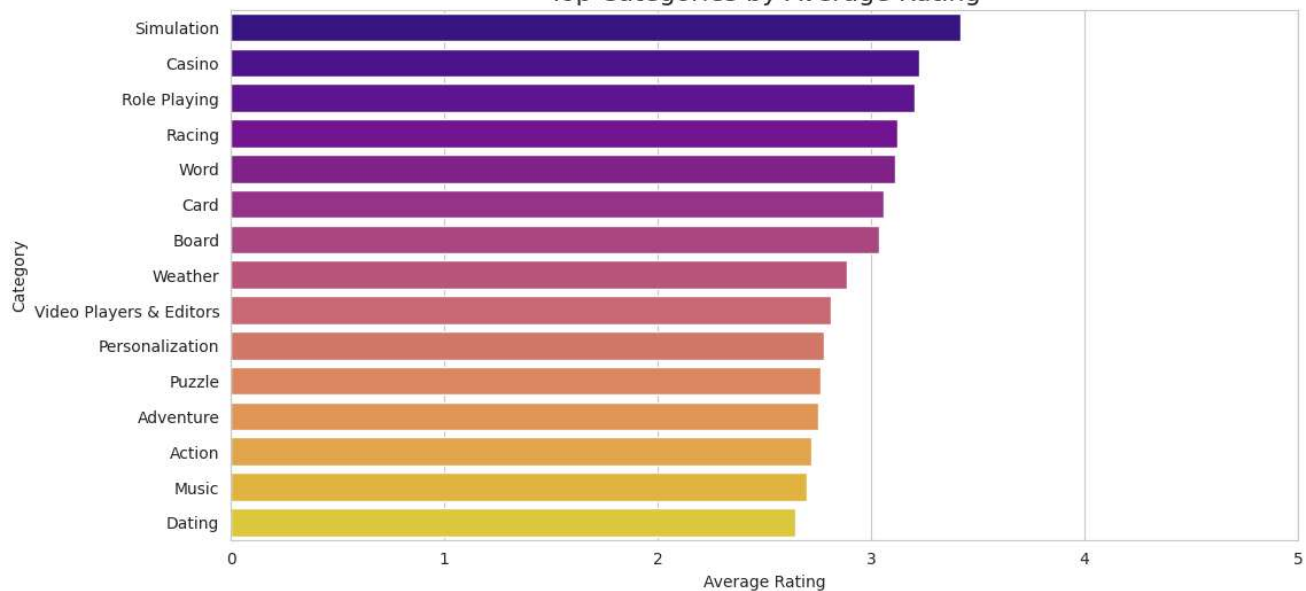
```

Distribution of App Ratings



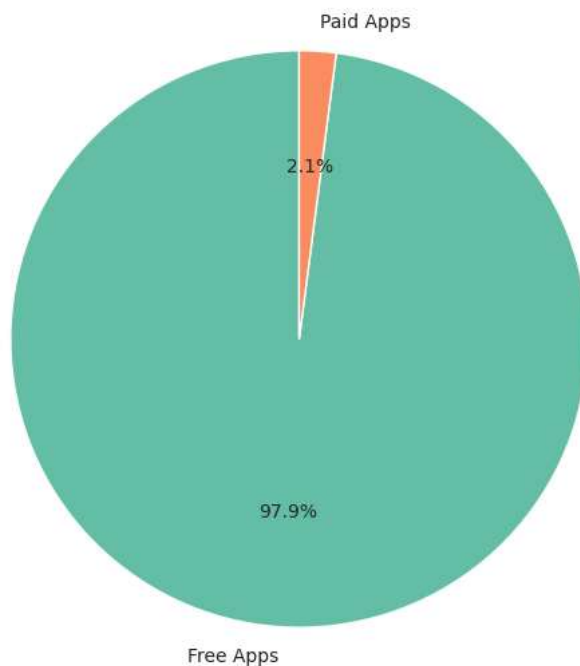
```
avg_rating = (df.groupby('Category')['Rating'].mean().sort_values(ascending=False).head(15))
plt.figure(figsize=(12,6))
sns.barplot(x=avg_rating.values,y=avg_rating.index,hue=avg_rating.index,palette='plasma',legend=False)
plt.title("Top Categories by Average Rating", fontsize=16)
plt.xlabel("Average Rating")
plt.ylabel("Category")
plt.xlim(0,5)
plt.show()
```

Top Categories by Average Rating



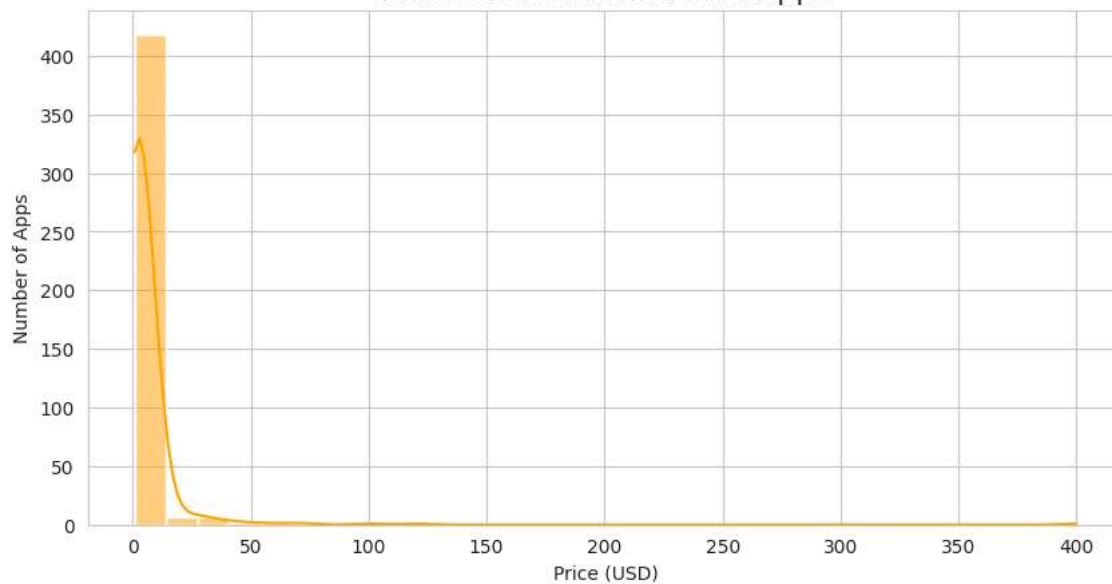
```
free_paid = df['Free'].value_counts()
labels = ['Free Apps', 'Paid Apps']
plt.figure(figsize=(7,7))
plt.pie(free_paid.values,labels=labels[:len(free_paid)],autopct='%1.1f%%',startangle=90,colors=['#66c2a5', '#fc8d62'])
plt.title("Free vs Paid Apps", fontsize=16)
plt.show()
```

Free vs Paid Apps



```
paid_apps = df[df['Free'] == False].copy()
paid_apps = paid_apps[paid_apps['Price'] > 0]
plt.figure(figsize=(10,5))
sns.histplot(paid_apps['Price'],bins=30,kde=True,color='orange')
plt.title("Price Distribution of Paid Apps", fontsize=16)
plt.xlabel("Price (USD)")
plt.ylabel("Number of Apps")
plt.show()
```

Price Distribution of Paid Apps

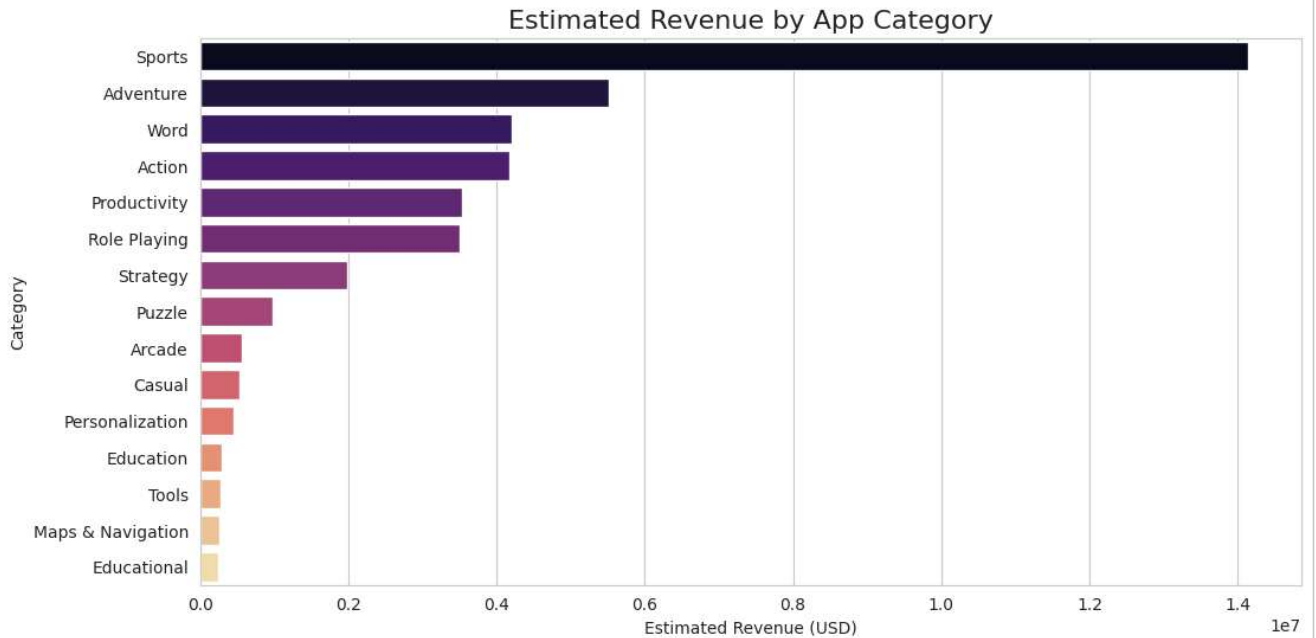


```
df['Estimated Revenue'] = np.where(
    df['Free'] == False,
    df['Price'] * df['Maximum Installs'],0)
```

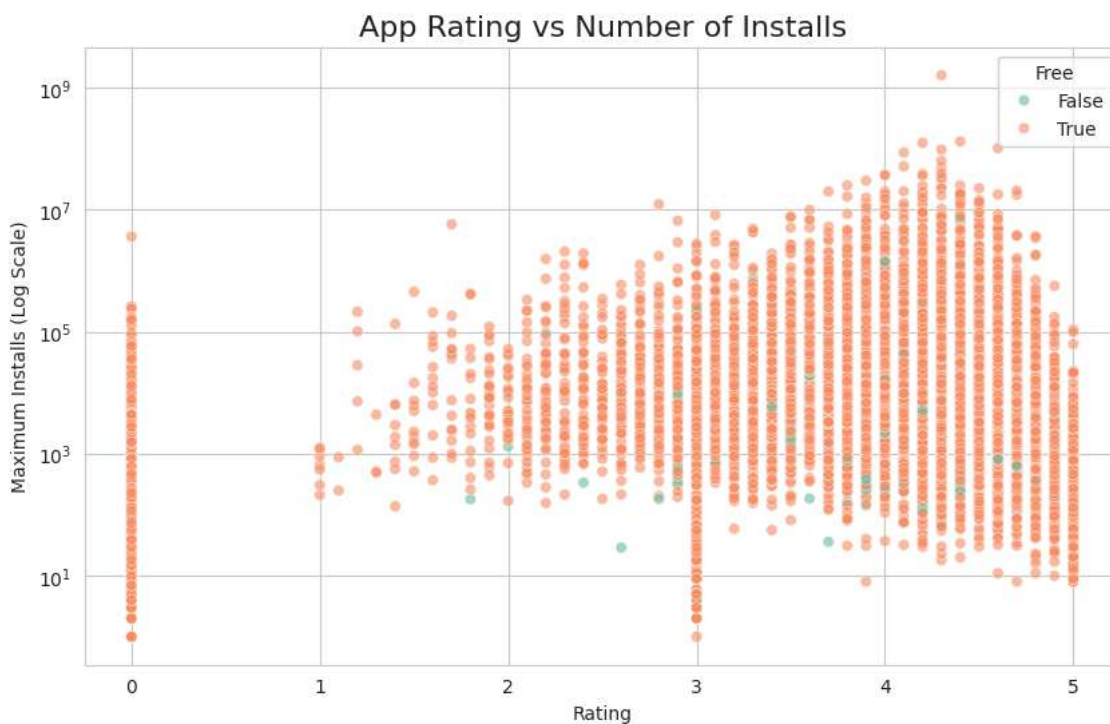
```
category_revenue = (
    df.groupby('Category')['Estimated Revenue'].sum().sort_values(ascending=False).head(15))
```

```
plt.figure(figsize=(12,6))
```

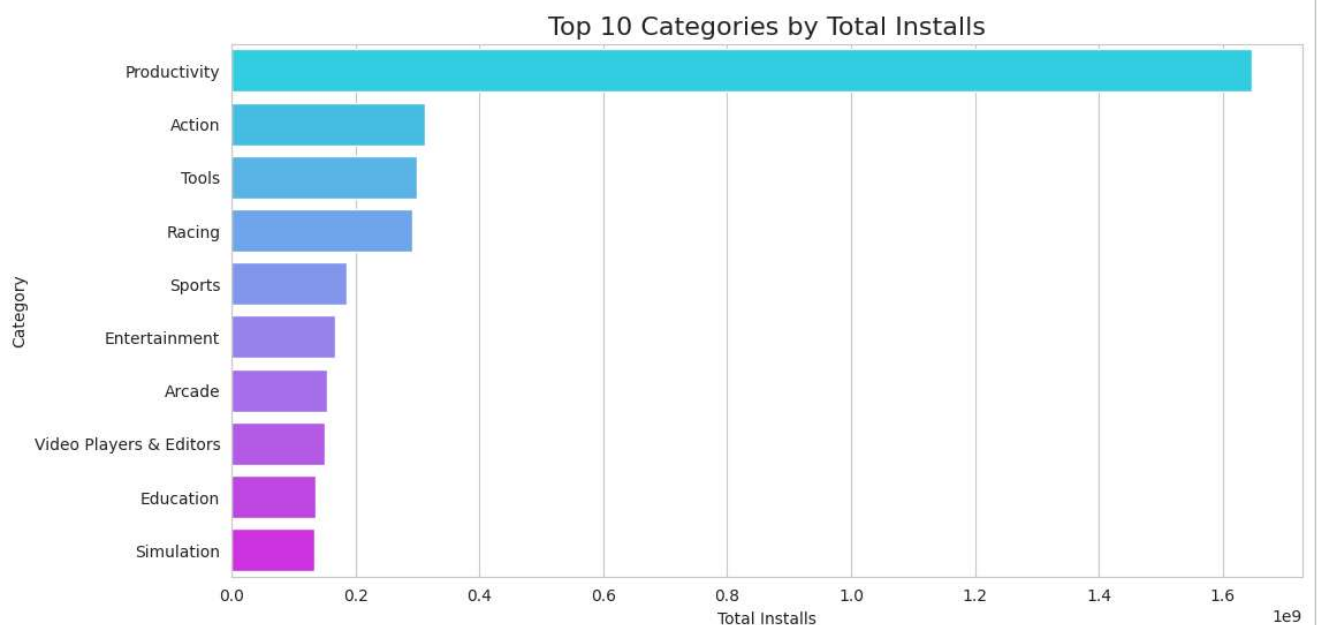
```
sns.barplot(x=category_revenue.values,y=category_revenue.index,hue=category_revenue.index,palette='magma',legend=False)
plt.title("Estimated Revenue by App Category", fontsize=16)
plt.xlabel("Estimated Revenue (USD)")
plt.ylabel("Category")
plt.show()
```



```
rating_install = df[
    (df['Rating'].notna()) &
    (df['Maximum Installs'] > 0)]
plt.figure(figsize=(10,6))
sns.scatterplot(data=rating_install,x='Rating',y='Maximum Installs',hue='Free',palette='Set2',alpha=0.6)
plt.yscale('log')
plt.title("App Rating vs Number of Installs", fontsize=16)
plt.xlabel("Rating")
plt.ylabel("Maximum Installs (Log Scale)")
plt.show()
```



```
category_installs = (  
    df.groupby('Category')['Maximum Installs']  
    .sum()  
    .sort_values(ascending=False)  
    .head(10)  
)  
plt.figure(figsize=(12,6))  
sns.barplot(  
    x=category_installs.values,  
    y=category_installs.index,  
    hue=category_installs.index,  
    palette='cool',  
    legend=False  
)  
plt.title("Top 10 Categories by Total Installs", fontsize=16)  
plt.xlabel("Total Installs")  
plt.ylabel("Category")  
plt.show()
```



Start coding or [generate](#) with AI.